

Universidad Técnica Estatal de Quevedo

Facultad de Ciencias de la Computación y Diseño Digital

Carrera de Ingeniería de Software



Modelado de Comunicación entre Procesos Distribuidos

Escenario de negocio: sistema de reservas de vuelos y hoteles

FORM-TL-01

Autores:

Jeremy Alexis Alvarez Párraga
Jhinson Stalyn Aucatoma Celorio
Carlos José Carpio Mendoza

Equipo D

Jueves 18 de Junio 2026

Repositorio GitHub

https://github.com/JhinsonAucatoma/Sistemas_distribuidos/tree/main/Trabajos_Grupales/Practica_experimental1

Índice

1. Objetivo	2
2. Marco teórico	2
2.1. Comunicación entre procesos en sistemas distribuidos	2
2.2. TCP y UDP: el mismo problema, dos soluciones distintas	3
2.3. Middleware: la capa que no debería depender del socket	3
2.4. Arquitectura cliente-servidor y concurrencia	3
3. Escenario de negocio: sistema de reservas de vuelos y hoteles	4
4. Diagrama de comunicación	5
4.1. Cómo se lee el flujo TCP	6
4.2. Cómo se lee el flujo UDP	6
5. Protocolo de comunicación y middleware: justificación	6
5.1. Por qué TCP para reservar y confirmar	6
5.2. Por qué UDP para consultar disponibilidad	7
5.3. Resumen de la decisión	7
6. Anexo: sustento técnico de la implementación en Java	7
6.1. El lado TCP	7
6.2. El lado UDP	8
7. Conclusiones	8

1. Objetivo

Cuando empezamos a revisar la consigna del taller nos dimos cuenta de que el enfoque inicial que le dimos al trabajo —documentar paso a paso una implementación en Java— no era lo que realmente se pedía. El ejercicio busca otra cosa: pensar primero el problema de comunicación a nivel de modelo, y recién después bajar a código si se quiere sustentar esa decisión. Por eso este documento parte de cero en la parte de modelado, aunque conserva el trabajo de sockets que ya habíamos hecho como evidencia de que el análisis no es solo teórico.

El objetivo central, entonces, es modelar la comunicación entre procesos distribuidos de un sistema de reservas de vuelos y hoteles: qué protocolo usar en cada operación, por qué, y qué capa de middleware tendría sentido colocar entre el cliente y el servidor para que esa decisión no quede atada directamente al socket.

De manera más puntual, nos propusimos:

- Elegir un escenario de negocio concreto (no genérico) y describir qué tipo de comunicación exige cada una de sus operaciones.
- Representar ese escenario en un diagrama, mostrando los procesos involucrados y el sentido de los mensajes entre ellos.
- Argumentar, con apoyo bibliográfico, por qué ciertas operaciones deben viajar por TCP y otras por UDP.
- Identificar qué modelo de middleware (RPC o MOM) encaja mejor con cada tipo de comunicación, en vez de asumir que todo pasa directamente por sockets crudos.
- Mostrar, a modo de respaldo, que la implementación de sockets que desarrollamos en la práctica anterior es coherente con este modelo.

2. Marco teórico

2.1. Comunicación entre procesos en sistemas distribuidos

Un sistema distribuido, en términos simples, es un conjunto de procesos que corren en máquinas distintas y que necesitan ponerse de acuerdo para que el usuario perciba un solo servicio coherente. Tanenbaum y van Steen describen esta coordinación como el problema central de la materia: lograr que componentes autónomos, sin memoria compartida, cooperen como si fueran uno solo [1]. La comunicación entre procesos (IPC) es justamente el mecanismo que hace posible ese acuerdo, y en la práctica casi siempre se apoya en el modelo cliente-servidor: un proceso pide algo y otro responde.

Lo interesante —y lo que en nuestra primera entrega no llegamos a discutir— es que no toda comunicación dentro de un mismo sistema tiene que viajar por el mismo canal ni de la misma forma. Un sistema real suele combinar varios tipos de comunicación según lo que cada operación necesite garantizar.

2.2. TCP y UDP: el mismo problema, dos soluciones distintas

A nivel de transporte, la decisión más básica es si se necesita una conexión confiable o no. TCP abre una conexión previa (el famoso *three-way handshake*), numera los segmentos, los retransmite si se pierden y los entrega en orden; esto tiene un costo en latencia, pero asegura que lo que el servidor recibe es exactamente lo que el cliente envió [2]. UDP, en cambio, no abre nada: simplemente empaqueta los datos en un datagrama y lo manda, sin esperar confirmación ni garantizar que llegue completo o en orden [2].

Stevens, en su libro clásico sobre programación de sockets, resume bien la consecuencia práctica de esto: la elección entre TCP y UDP no es una cuestión de gustos, sino de qué puede tolerar perder la aplicación [3]. Si un mensaje de confirmación de pago se pierde y nadie se entera, el problema es serio. Si se pierde un mensaje de “hay 3 asientos disponibles” y el cliente simplemente vuelve a preguntar medio segundo después, no pasa nada grave. Esa lógica es la que terminamos usando para justificar el diseño del escenario.

2.3. Middleware: la capa que no debería depender del socket

Algo que omitimos por completo en la entrega anterior es la idea de middleware. Coulouris y otros lo definen como el software que se ubica entre el sistema operativo y la aplicación, encargado de esconder los detalles de la red para que el programador no tenga que pensar en sockets todo el tiempo [4]. En la práctica esto se traduce en un par de modelos que conviene distinguir:

- **RPC (Remote Procedure Call):** hace que invocar algo en un proceso remoto se vea, desde el código, casi igual que llamar una función local. Es síncrono: el que llama espera la respuesta antes de seguir [4].
- **RMI (Remote Method Invocation):** es la versión de RPC para objetos, propia de Java; en vez de invocar funciones se invocan métodos de objetos remotos [5].
- **MOM (Message-Oriented Middleware):** en lugar de una llamada directa, los procesos se comunican dejando mensajes en una cola o publicándolos en un tópico, y quien los procesa lo hace de forma asíncrona, sin que el emisor tenga que esperar [4].

Para este trabajo decidimos asociar RPC a las operaciones que necesitan una respuesta inmediata y confirmada (reservar, confirmar, cancelar) y MOM a las que son más bien notificaciones o consultas frecuentes (disponibilidad). No es la única forma válida de modelarlo, pero nos parece la que mejor refleja cómo funcionan sistemas reales de este tipo.

2.4. Arquitectura cliente-servidor y concurrencia

Un detalle que no conviene pasar por alto es que ninguno de estos modelos funciona bien si el servidor solo puede atender a un cliente a la vez. En un sistema de reservas reales hay, en cualquier momento dado, decenas o cientos de personas consultando

disponibilidad o tratando de confirmar una compra, y el servidor tiene que poder atenderlas sin que una bloquee a las demás.

La solución más habitual —y la que ya habíamos usado en la implementación de la práctica anterior— es que el proceso servidor delegue cada conexión entrante a un hilo independiente, de modo que mientras un hilo procesa la reserva de un cliente, otro hilo puede estar atendiendo la consulta de disponibilidad de otro. Tanenbaum y van Steen señalan que esta es, de hecho, una de las razones por las que el modelo cliente-servidor sigue siendo tan usado: permite escalar la atención de solicitudes sin que cada cliente necesite saber nada de los demás [1]. En el caso de UDP la concurrencia es algo distinta, porque no hay una conexión que mantener abierta por cliente, pero el servidor igual necesita poder procesar ráfagas de datagramas sin que una consulta lenta retrase a las siguientes.

3. Escenario de negocio: sistema de reservas de vuelos y hoteles

El escenario que elegimos es uno bastante conocido para cualquiera que haya comprado un pasaje o reservado un hotel en línea: una plataforma donde varios usuarios, al mismo tiempo, están buscando disponibilidad, reservando asientos o habitaciones, y esperando una confirmación de que su pago se procesó correctamente.

Lo que hace interesante a este escenario —y por lo que finalmente lo elegimos en lugar de uno más simple— es que mezcla dos necesidades de comunicación que son casi opuestas:

Por un lado está la reserva en sí. Cuando alguien aprieta “confirmar reserva”, esa operación no puede fallar a medias: si el sistema le cobra al cliente pero no le asigna el asiento, o si dos personas terminan con la misma habitación porque el mensaje de “ya está ocupada” se perdió en el camino, el negocio tiene un problema serio. Esta es una operación que necesita confiabilidad por encima de todo.

Por otro lado está la consulta de disponibilidad, que es la operación que más veces se ejecuta. Cada vez que alguien cambia las fechas de búsqueda, el sistema tiene que volver a preguntar “¿cuántos asientos quedan?”. Esto pasa muchas veces por segundo en un sistema con tráfico real, y no tiene sentido pagar el costo de una conexión TCP completa para cada una de esas consultas. Además, si una respuesta se pierde, el cliente simplemente puede volver a pedirla sin que eso afecte la integridad de ningún dato.

Esa diferencia es, en el fondo, la razón por la que el escenario se modela con dos canales de comunicación separados en vez de uno solo: TCP para todo lo que implica reservar y confirmar, UDP para todo lo que implica consultar disponibilidad en tiempo real. Plataformas reales como las de aerolíneas o cadenas hoteleras funcionan, a grandes rasgos, bajo esta misma lógica de separar lo transaccional de lo consultivo.

Un caso de uso concreto

Para que quede más claro cómo se conectan estas dos partes, conviene seguir un ejemplo de principio a fin. Una usuaria abre la aplicación para buscar vuelos entre Quito y Madrid para una fecha determinada. Mientras escribe y ajusta los filtros (fechas, número de pasajeros, escalas), la aplicación envía varias consultas de disponibilidad al servidor; cada una de esas consultas usa UDP, porque lo único que importa es la respuesta más reciente, y si una se pierde no tiene sentido reintentarla con cuidado: simplemente se manda otra.

Cuando la usuaria finalmente elige un vuelo y decide comprarlo, la aplicación cambia de canal: abre una conexión TCP y envía la solicitud de reserva. Recién en este punto el sistema necesita la garantía de que el mensaje llegue, de que el servidor confirme explícitamente si el asiento sigue disponible y de que, si todo salió bien, la reserva quede registrada antes de decirle a la usuaria que su compra se completó. Este pequeño recorrido —muchas consultas livianas por UDP y, al final, una operación crítica por TCP— es, en esencia, el comportamiento que el diagrama de la siguiente sección intenta representar.

4. Diagrama de comunicación

El diagrama de la Figura 1 fue elaborado en draw.io y se entrega también como archivo fuente editable (.drawio) junto con este documento, para que se pueda modificar o ampliar si el docente lo pide. Muestra al cliente, la capa de middleware (separada en su componente RPC y su componente MOM) y los dos procesos servidores, junto con la base de datos donde finalmente se persiste la información.

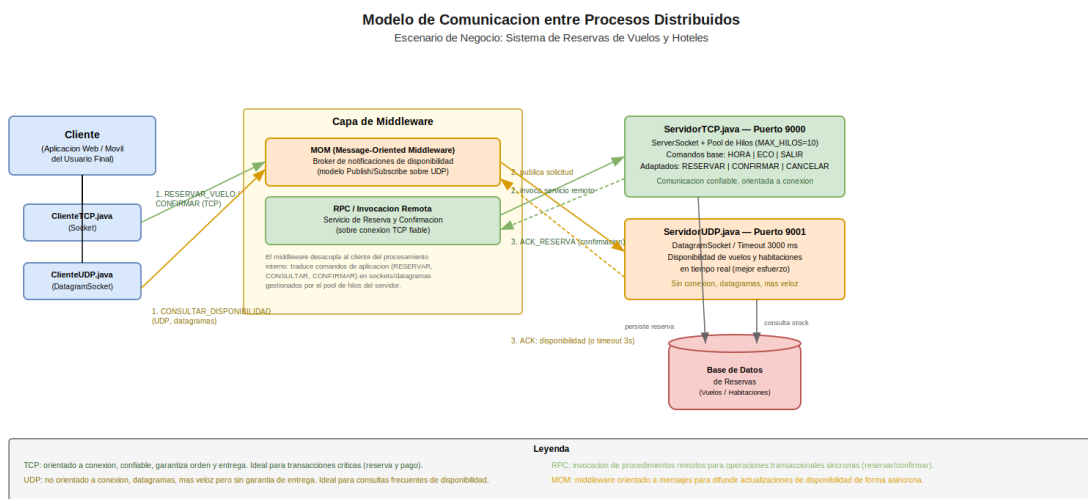


Figura 1: Diagrama de comunicación entre procesos del sistema de reservas de vuelos y hoteles.

4.1. Cómo se lee el flujo TCP

La idea de este flujo es que todo lo que implica comprometer un recurso (un asiento, una habitación) pase por un canal confiable de punta a punta:

1. El cliente, a través de `ClienteTCP.java`, abre una conexión y envía el comando `RESERVAR_VUELO` o `CONFIRMAR`.
2. La capa RPC del middleware traduce ese comando en una invocación al servicio remoto que expone `ServidorTCP.java` en el puerto 9000.
3. El servidor procesa la solicitud, guarda la reserva en la base de datos y devuelve un `ACK_RESERVA` por el mismo canal, confirmando que la operación se completó.

4.2. Cómo se lee el flujo UDP

Aquí la lógica cambia: no hace falta abrir nada antes de preguntar, y si la respuesta no llega, el cliente puede simplemente intentar de nuevo.

1. El cliente envía `CONSULTAR_DISPONIBILIDAD` mediante `ClienteUDP.java`, como un datagrama suelto, sin conexión previa.
2. El middleware MOM recibe esa solicitud y se la pasa a `ServidorUDP.java` (puerto 9001), que consulta cuántos asientos u habitaciones quedan.
3. El servidor responde con la disponibilidad actual. Si la respuesta no llega dentro de los 3 segundos configurados como tiempo máximo de espera, el cliente captura una excepción de timeout y puede reintentar.

5. Protocolo de comunicación y middleware: justificación

5.1. Por qué TCP para reservar y confirmar

Ya lo mencionamos antes pero conviene dejarlo explícito acá: una reserva de vuelo o de hotel es, en esencia, una transacción. No alcanza con que el mensaje “llegue rápido”; tiene que llegar completo, en orden, y el sistema necesita saber con certeza si la operación se concretó o no. TCP ofrece justamente eso mediante control de flujo, confirmación de recepción y retransmisión automática de los segmentos perdidos [2], [3].

Sobre ese canal colocamos conceptualmente un middleware de tipo RPC, porque la operación de reservar se comporta como una llamada a un procedimiento remoto: el cliente envía una solicitud, se queda esperando, y solo continúa cuando recibe una respuesta explícita de éxito o fallo. No tendría sentido modelarla como un mensaje asíncrono, porque el usuario necesita saber en el momento si su reserva se concretó.

5.2. Por qué UDP para consultar disponibilidad

La consulta de disponibilidad, en cambio, es de alta frecuencia y de bajo riesgo si algo se pierde. Un usuario puede estar refrescando la búsqueda de vuelos varias veces en pocos segundos, y cada una de esas consultas no necesita el overhead de abrir y cerrar una conexión TCP completa. UDP permite enviar esa consulta como un datagrama suelto, sin negociación previa, lo que reduce notablemente la latencia percibida [2].

El middleware asociado en este caso es MOM, porque el patrón de comunicación se acerca más a un esquema de publicación de estado: el servidor podría, en una versión más completa del sistema, difundir actualizaciones de disponibilidad a varios clientes interesados sin que cada uno tenga que pedirla explícitamente cada vez [4].

5.3. Resumen de la decisión

Cuadro 1: Síntesis de la decisión de protocolo y middleware por tipo de operación.

Operación	Protocolo	Middleware	Criterio dominante
Reservar, confirmar o cancelar	TCP	RPC	Confiabilidad transaccional
Consultar disponibilidad	UDP	MOM	Velocidad y tolerancia a la pérdida

6. Anexo: sustento técnico de la implementación en Java

Esta sección no es el entregable principal del taller, sino un respaldo de que el análisis anterior no quedó solo en el papel: ya habíamos construido (en la práctica experimental previa) una implementación funcional de sockets TCP y UDP en Java que reproduce, a pequeña escala, el comportamiento descrito en el modelo.

6.1. El lado TCP

Las clases `ServidorTCP` y `ClienteTCP`, dentro del paquete `ec.edu.uteq.distribuidas.tcp`, abren un `ServerSocket` en el puerto 9000 y usan un pool fijo de 10 hilos para poder atender varios clientes al mismo tiempo sin que uno bloquee a los demás, algo bastante parecido a lo que haría falta en un servidor de reservas con tráfico real.

```
1 private static final int PUERTO = 9000;
2 private static final int MAX_HILOS = 10;
3
4 while (true) {
5     Socket cliente = servidor.accept();
6     // se asigna un hilo independiente (ManejadorCliente)
7 }
```

Listing 1: Configuración del servidor TCP con pool de hilos.

Los comandos que probamos en esa práctica (HORA, ECO, SALIR) son, en esencia, una versión simplificada de lo que en el modelo de negocio serían CONSULTAR_HORA_SERVIDOR, CONFIRMAR y CANCELAR: la estructura del código —pedir algo, recibir una respuesta confirmada por el mismo canal— es la misma.

6.2. El lado UDP

Las clases `ServidorUDP` y `ClienteUDP`, en `ec.edu.uteq.distribuidas.udp`, usan un `DatagramSocket` sobre el puerto 9001 y un tiempo máximo de espera de 3000 ms, justamente para no dejar al cliente esperando indefinidamente si la respuesta no llega, algo que en el modelo corresponde directamente al flujo de consulta de disponibilidad.

```
1 private static final int PUERTO = 9001;
2 socket.setSoTimeout(TIMEOUT); // 3000 ms
3
4 catch (SocketTimeoutException e) {
5     System.out.println("Timeout: no se recibió respuesta en "
6         + TIMEOUT + "ms");
7 }
```

Listing 2: Manejo de timeout en el cliente UDP.

El código completo, junto con las capturas de ejecución de ambos servidores y clientes, sigue disponible en el mismo repositorio de GitHub que entregamos antes: https://github.com/JhinsonAucatoma/Sistemas_distribuidos/tree/main/Trabajos_Grupales/Practica_experimental1

7. Conclusiones

Volviendo sobre lo que aprendimos al rehacer este trabajo: el error inicial no fue técnico sino de enfoque. Sabíamos bastante bien cómo funcionan TCP y UDP, pero documentamos una implementación cuando lo que se pedía era modelar una decisión de arquitectura antes de escribir cualquier línea de código.

Pensar el escenario de reservas en términos de “qué puede fallar y qué no” fue lo que terminó ordenando todo lo demás: una vez que separamos las operaciones transaccionales de las consultivas, la elección de protocolo y de middleware dejó de ser arbitraria y pasó a tener una justificación concreta detrás. También nos quedó más claro que el middleware no es un detalle decorativo del diagrama: es la pieza que evita que la lógica de negocio quede pegada para siempre a un socket específico.

Referencias

- [1] A. S. Tanenbaum y M. van Steen, *Distributed Systems: Principles and Paradigms*, 3.^a ed. Boston, MA, EE. UU.: Pearson, 2023.

- [2] J. F. Kurose y K. W. Ross, *Computer Networking: A Top-Down Approach*, 8.^a ed. Hoboken, NJ, EE. UU.: Pearson, 2021.
- [3] W. R. Stevens, B. Fenner y A. M. Rudoff, *UNIX Network Programming, Volume 1: The Sockets Networking API*, 3.^a ed. Boston, MA, EE. UU.: Addison-Wesley, 2004.
- [4] G. Coulouris, J. Dollimore, T. Kindberg y G. Blair, *Distributed Systems: Concepts and Design*, 5.^a ed. Boston, MA, EE. UU.: Addison-Wesley, 2012.
- [5] Oracle, *Java Remote Method Invocation (RMI) Documentation*, Oracle Help Center, [En línea], 2024. dirección: <https://docs.oracle.com/javase/8/docs/technotes/guides/rmi/>.